PHD PROGRAM

Applied Data Science & Artificial Intelligence

UNIVERSITÀ DEGLI STUDI DI TRIESTE

# SHAP method

# SHAP (SHapley Additive exPlanations)

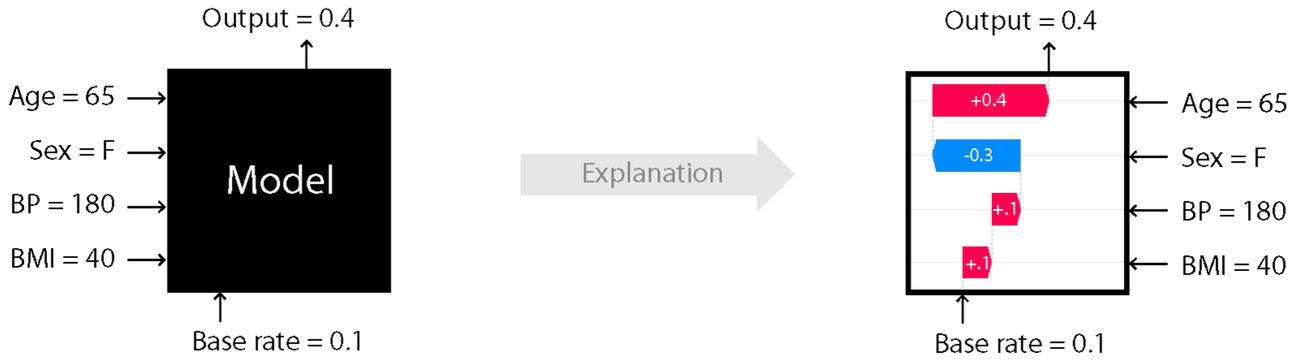
- Additive feature attribution method

$$f(\boldsymbol{x}^*) = \phi_0 + \sum_{j=1}^{M} \phi_j^*$$

where $\phi_0 = E_{p(\boldsymbol{x})}[\, f(\boldsymbol{x}) \,]$

$\phi_j$ = “contribution of feature j”

- Local, post-hoc explanations
- Model agnostic
- Feature attributions are **Shapley values**




Lundberg, Lee. A unified approach to interpreting model predictions, *NEURIPS* (2017)

# Shapley values

Cooperative game theory concept to assign payouts to players, depending on their contribution towards the total payout.

In the explanation setting, features are the players and the prediction is the total payout

→ we **distribute among the features the difference between the prediction and the average prediction**

# Shapley values: formal definition

A team of players $\mathcal{M}$ = {1,2,...M} tries to maximize a payout v(M).

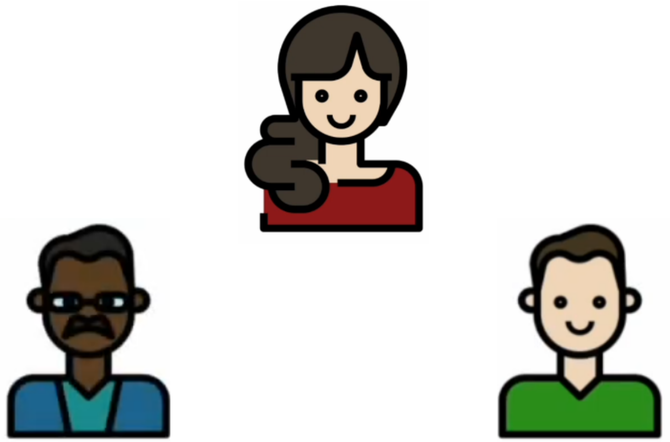
The Shapley value for each player is a weighted average over all his possible contributions:

$$\phi_j = \sum_{\mathcal{S} \subseteq \mathcal{M} \setminus \{j\}} \frac{|\mathcal{S}|!(M - |\mathcal{S}| - 1)!}{M!} (v(\mathcal{S} \cup \{j\}) - v(\mathcal{S})), \quad j = 1, \ldots, M$$

*v(S)* → **payout** the members of S can obtain by cooperation

*v(S ∪ {j}) - v(S)* → contribution of *j* to subset *S*

# Shapley values: example



$$\phi_j = \sum_{\mathcal{S} \subseteq \mathcal{M} \setminus \{j\}} \frac{|\mathcal{S}|!(M - |\mathcal{S}| - 1)!}{M!} (v(\mathcal{S} \cup \{j\}) - v(\mathcal{S})), \quad j = 1, \ldots, M$$

$$\phi_1 = \frac{1}{3}\left(v(\{1,2,3\}) - v(\{2,3\})\right) + \frac{1}{6}\left(v(\{1,2\}) - v(\{2\})\right) + \frac{1}{6}\left(v(\{1,3\}) - v(\{3\})\right) + \frac{1}{3}\left(v(\{1\}) - v(\emptyset)\right)$$

# From Shapley values to SHAP

How to evaluate the model on **subsets** of features?

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} [f_x(S \cup \{i\}) - f_x(S)]$$

One possible definition of $f_x(S)$ is as a **conditional (observational)** expectation value:

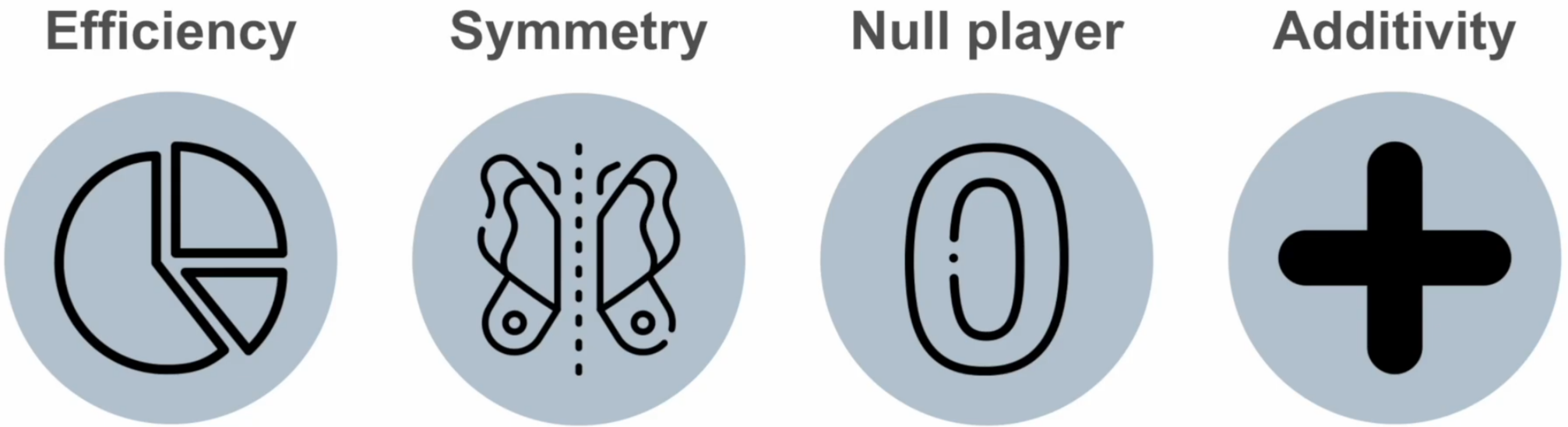
$$f_x(S) = \mathrm{E}[f(\boldsymbol{x}) | \boldsymbol{x}_S = \boldsymbol{x}_S^*]$$

meaning that the expected value is computed with respect to $p(\boldsymbol{x}_{\mathcal{M} \setminus \mathcal{S}} \mid \boldsymbol{x}_{\mathcal{S}})$

"SHAP values are the Shapley values for a chosen coalition-value function, such as the conditional expectation function of the original model. Different choices give different SHAP values."

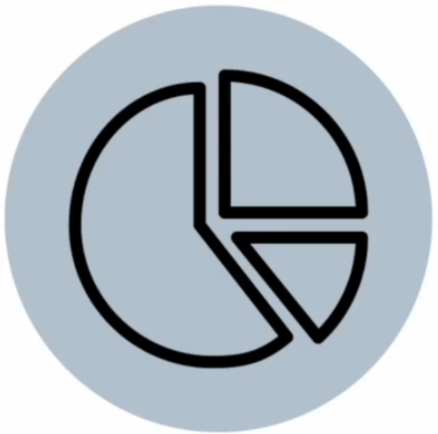
# SHAP values properties

Shapley values → **unique** solution in the class of additive feature attribution methods with some desirable properties



# SHAP values properties

## Efficiency

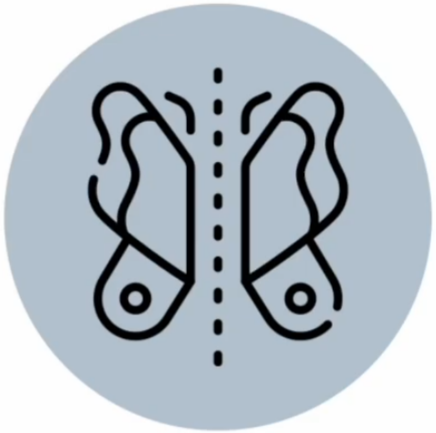


The sum of the Shapley values features is equal to the difference between the prediction and the global average prediction:

$$\sum_{j=1}^{M} \phi_j = f(\boldsymbol{x}^*) - \mathrm{E}[f(\boldsymbol{x})]$$

# SHAP values properties

## Symmetry

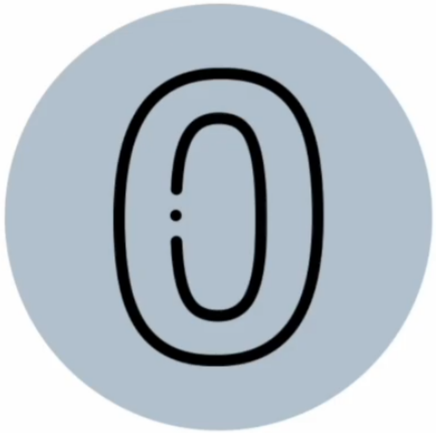


If two features contribute equally to all possible coalitions, then their Shapley values are identical

$$v(\mathcal{S} \cup \{i\}) = v(\mathcal{S} \cup \{j\}) \quad \Rightarrow \quad \phi_i = \phi_j$$

# SHAP values properties

## Null player



If a feature has zero contribution for all subsets of features, then its Shapley value is 0.

$$v(\mathcal{S} \cup \{j\}) = v(\mathcal{S}) \quad \Rightarrow \quad \phi_j = 0$$

# SHAP values properties




The Shapley value for a sum of functions is the sum of the Shapley value of the individual functions.

$$\phi_i(v + w) = \phi_i(v) + \phi_i(w)$$

→ useful for **ensemble** models

# KernelSHAP

Kernel SHAP → method to **estimate** Shapley values, introducing two approximations:

1. The sum over all the subsets involves $M2^{M-1}$ terms → computationally intractable

   → Kernel SHAP estimates Shapley values using **sampled subsets** of $\mathcal{M}$

   → Kernel SHAP relies on proving that Shapley values are the solution of a LIME regression problem over the boolean encoding of subsets of $\mathcal{M}$, using a special kernel.

Covert, Ian, and Su-In Lee. "Improving kernelshap: Practical shapley value estimation via linear regression." (2020).

# KernelSHAP

- Sample coalition vectors $\mathbf{z}'_k \in \{0,1\}^M, \quad k \in \{1,\ldots,K\}$ (1 = feature present in coalition, 0 = feature absent).
- Get prediction for each $\mathbf{z}'_k$ by first converting $\mathbf{z}'_k$ to the original feature space and then applying model $\hat{f}$ : $\hat{f}(h_{\mathbf{x}}(\mathbf{z}'_k))$.
- Compute the weight for each coalition $\mathbf{z}'_k$ with the SHAP kernel.
- Fit weighted linear model.
- Return Shapley values $\phi_k$, the coefficients from the linear model.

Covert, Ian, and Su-In Lee. "Improving kernelshap: Practical shapley value estimation via linear regression." (2020).

# KernelSHAP

- Sample coalition vectors $\mathbf{z}'_k \in \{0,1\}^M, \quad k \in \{1,\dots,K\}$ (1 = feature present in coalition, 0 = feature absent).
- Get prediction for each $\mathbf{z}'_k$ by first converting $\mathbf{z}'_k$ to the original feature space and then applying model $\hat{f}$ : $\hat{f}(h_{\mathbf{x}}(\mathbf{z}'_k))$.
- Compute the weight for each coalition $\mathbf{z}'_k$ with the SHAP kernel.
- Fit weighted linear model.
- Return Shapley values $\phi_k$, the coefficients from the linear model.

$$\hat{f}(h_x(z')) = \mathbb{E}_{X_{\bar{S}}}\left[\hat{f}(x_S, X_{\bar{S}})\right]$$

$$\pi_{\mathbf{x}}(\mathbf{z}') = \frac{(M-1)}{\binom{M}{|\mathbf{z}'|}|\mathbf{z}'|(M-|\mathbf{z}'|)}$$

$$g(\mathbf{z}') = \phi_0 + \sum_{j=1}^{M} \phi_j z'_j \qquad L(\hat{f}, g, \pi_{\mathbf{x}}) = \sum_{\mathbf{z}' \in \mathbf{Z}} [\hat{f}(h_{\mathbf{x}}(\mathbf{z}')) - g(\mathbf{z}')]^2 \pi_{\mathbf{x}}(\mathbf{z}')$$

Covert, Ian, and Su-In Lee. "Improving kernelshap: Practical shapley value estimation via linear regression." (2020).

# KernelSHAP

2. The conditional distribution is difficult to estimate

$$E[f(\boldsymbol{x})|\boldsymbol{x}_S = \boldsymbol{x}_S^*] = \int f(\boldsymbol{x}_{\bar{S}}, \boldsymbol{x}_S^*)\, p(\boldsymbol{x}_{\bar{S}}|\boldsymbol{x}_S = \boldsymbol{x}_S^*) d\boldsymbol{x}_{\bar{S}}$$

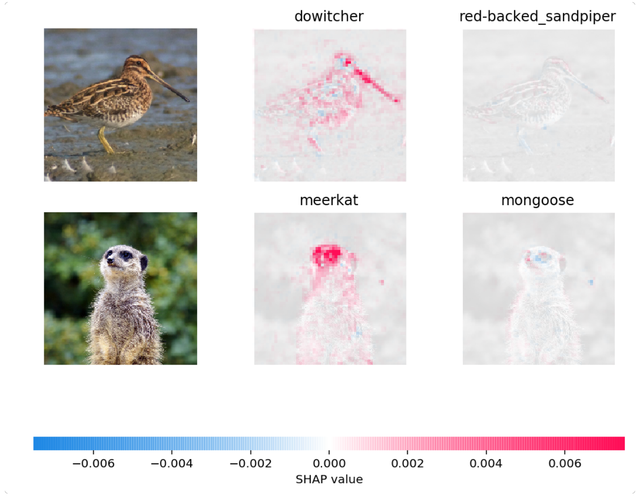
→ Kernel SHAP **breaks feature dependence** between retained and missing features and approximates the conditional distribution with the marginal distribution

$$p(\boldsymbol{x}_{\bar{S}}|\boldsymbol{x}_S = \boldsymbol{x}_S^*) \quad \sim \quad p(\boldsymbol{x}_{\bar{S}})$$

# Non-agnostic SHAP algorithms

- **TreeSHAP**
  Fast SHAP value estimation methods specific to trees and ensembles of trees.
  Implementations for *XGBoost*, *LightGBM*, *CatBoost*, *scikit-learn* and *pyspark* tree models.

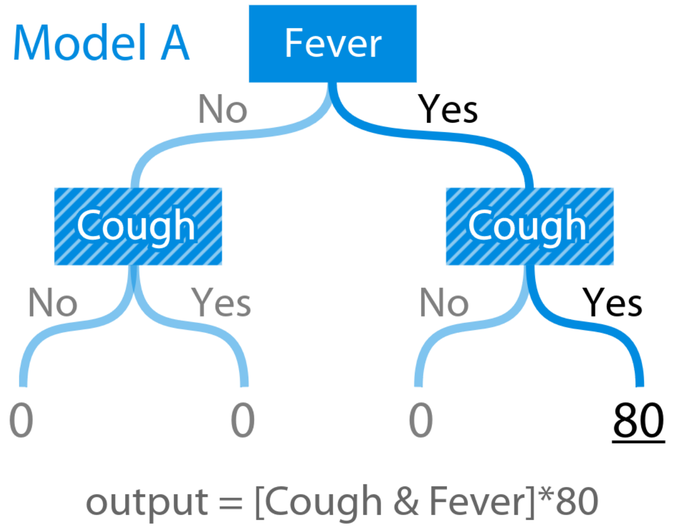
- **DeepSHAP**
  Efficient computation of SHAP values for neural networks, based on DeepLIFT.



# TreeSHAP

The SHAP value for a feature is the average change in model output by conditioning on that feature when introducing features one at a time over all feature orderings. This is challenging to compute.

- Complexity in trees is $O(LT2^M)$, where $T$ is the number of trees in the tree ensemble model, $L$ is maximum number of leaves in any tree and $M$ is the number of features
- In the TreeSHAP paper, the authors propose a modified version of this algorithm that keeps track of the number of subsets S that flow into each node of the tree. The modified algorithm has a computational complexity of $O(LTD^2)$ where $D$ is the max depth of the tree.




Lundberg, Scott M., Gabriel G. Erion, and Su-In Lee. "Consistent individualized feature attribution for tree ensembles." (2018).

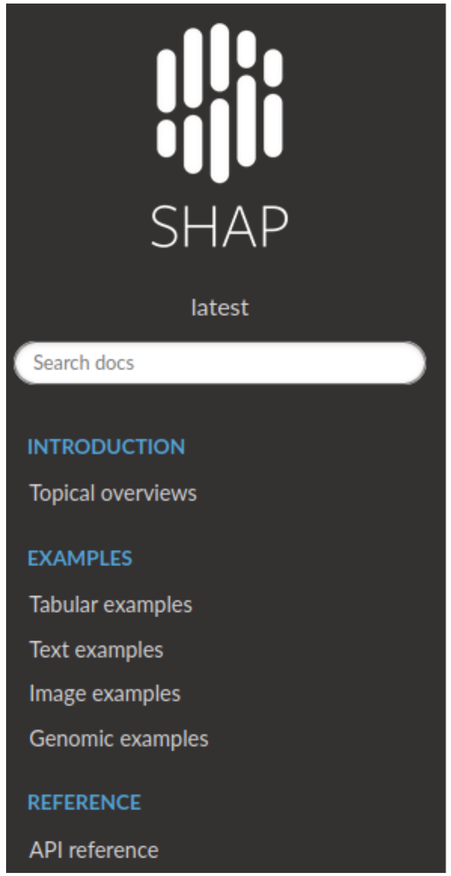
# DeepSHAP

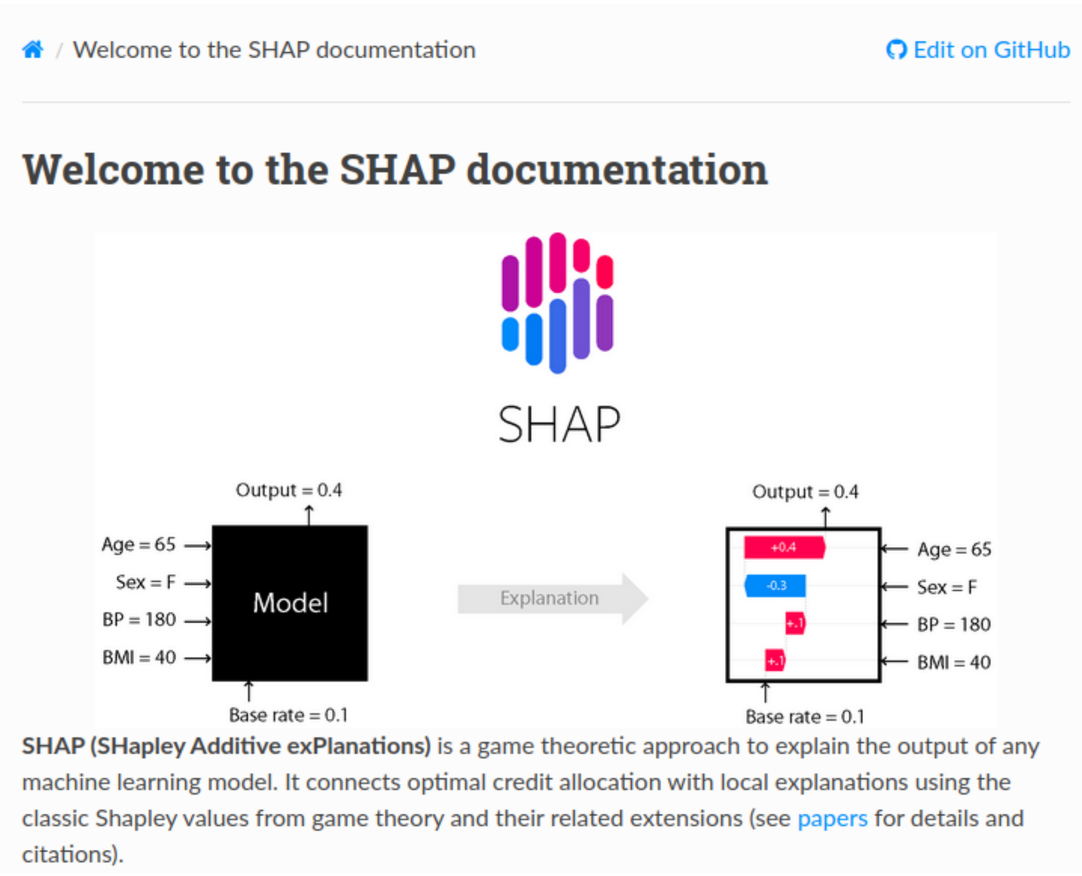
Deep SHAP is an enhanced version of the DeepLIFT algorithm meant to approximate SHAP values for deep learning models.

- Similarly to Kernel SHAP, the conditional expectations of SHAP values are approximated using a selection of background samples.
- DeepLIFT (Deep Learning Important FeaTures) is a method for decomposing the output prediction of a neural network on a specific input by backpropagating the contributions of all neurons in the network to every feature of the input. DeepLIFT compares the activation of each neuron to its ‘reference activation’ and assigns contribution scores according to the difference. Scores can be computed efficiently in a single backward pass.
- Per node attribution rules in DeepLIFT can be used to approximate Shapley values. Deep SHAP estimates approximate SHAP values by integrating over many background samples and such that they sum up to the difference between the expected model output on the passed background samples and the current model output (f(x) - E[f(x)]).

Shrikumar, Avanti, Peyton Greenside, and Anshul Kundaje. "Learning important features through propagating activation differences." International conference on machine learning. PMLR, 2017.

# SHAP package



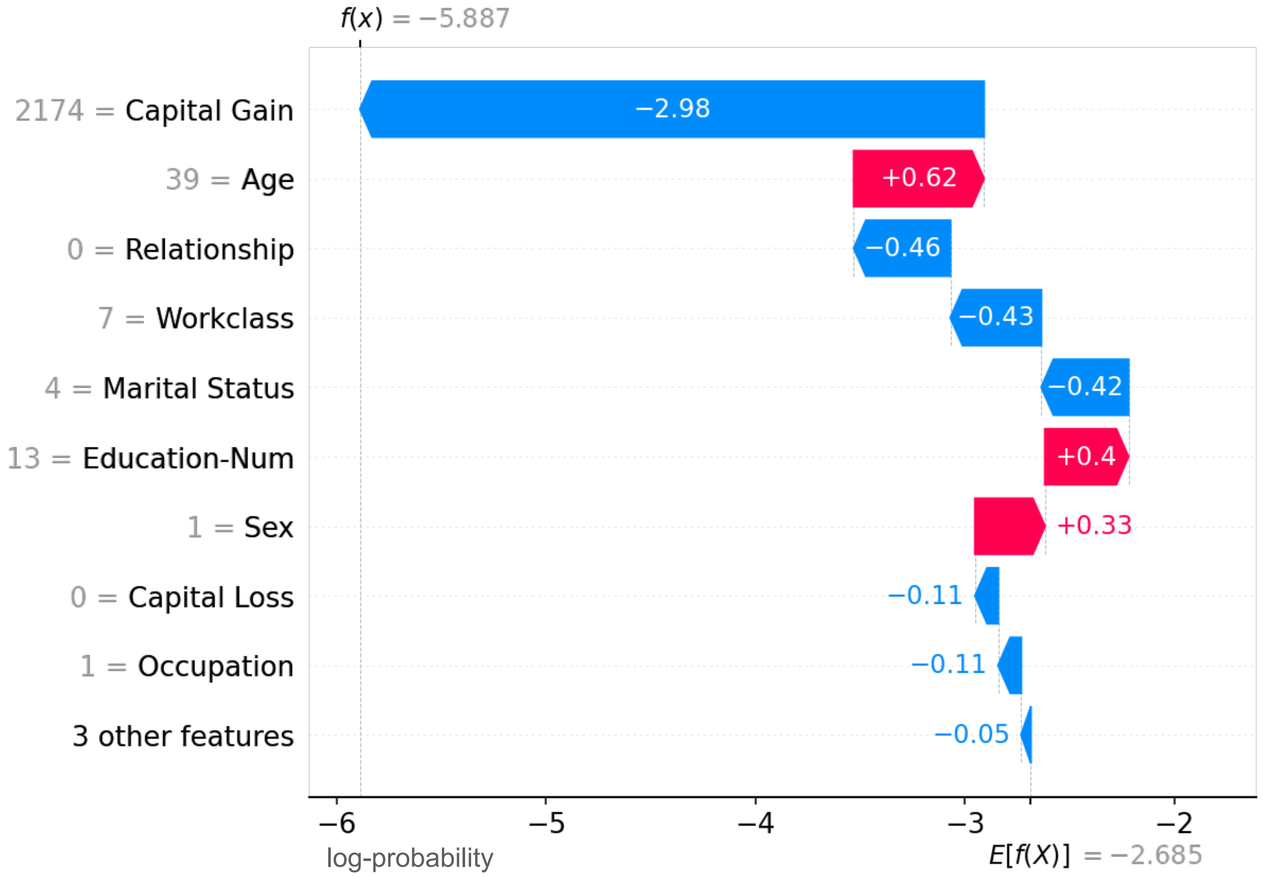




https://shap.readthedocs.io/en/latest/index.html
https://github.com/shap/shap

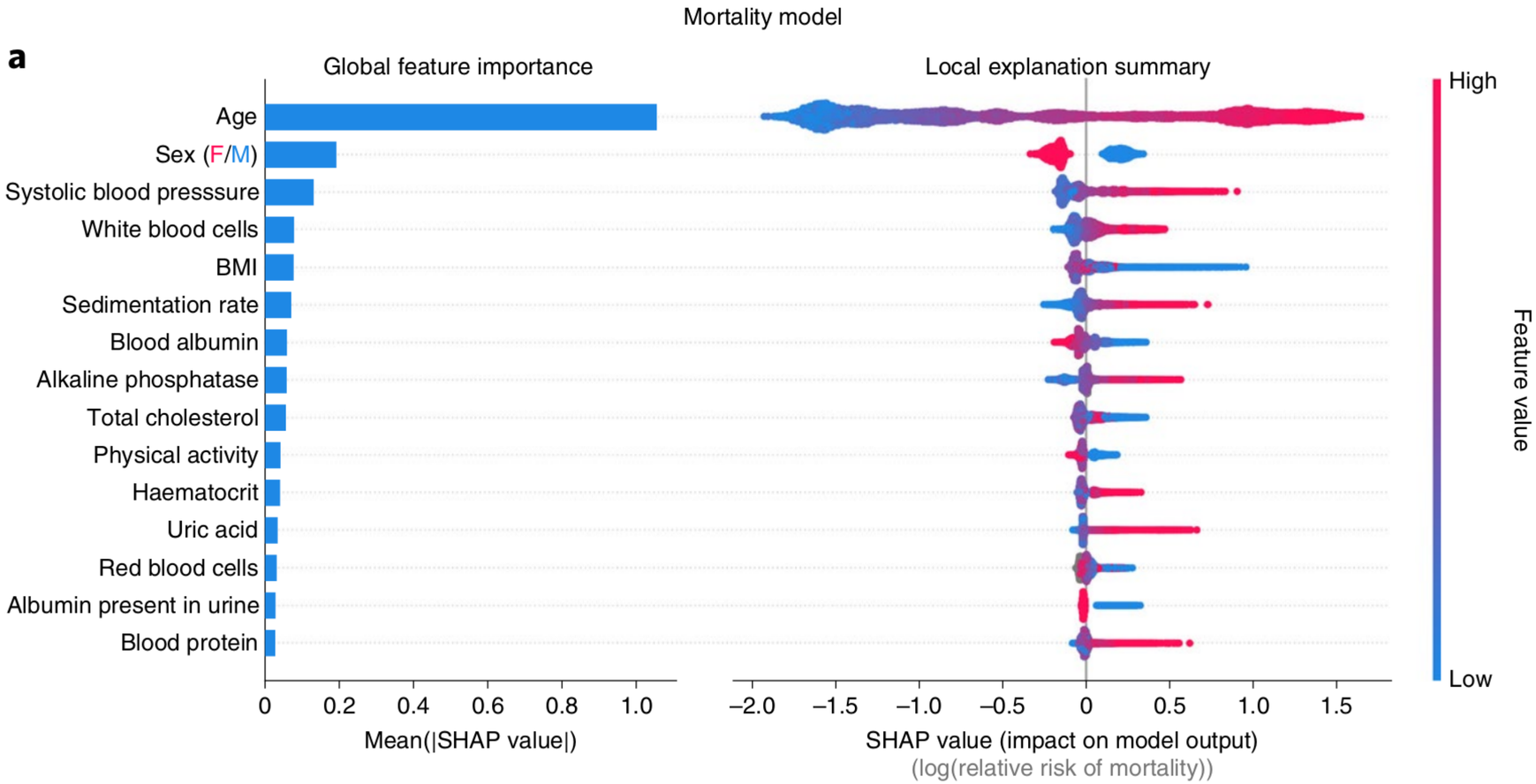
# Local explanations

**Waterfall plots** → display explanations for individual predictions

<u>Example</u>
Predict whether income exceeds $50K/yr based on census data



# Global explanations (mortality model)

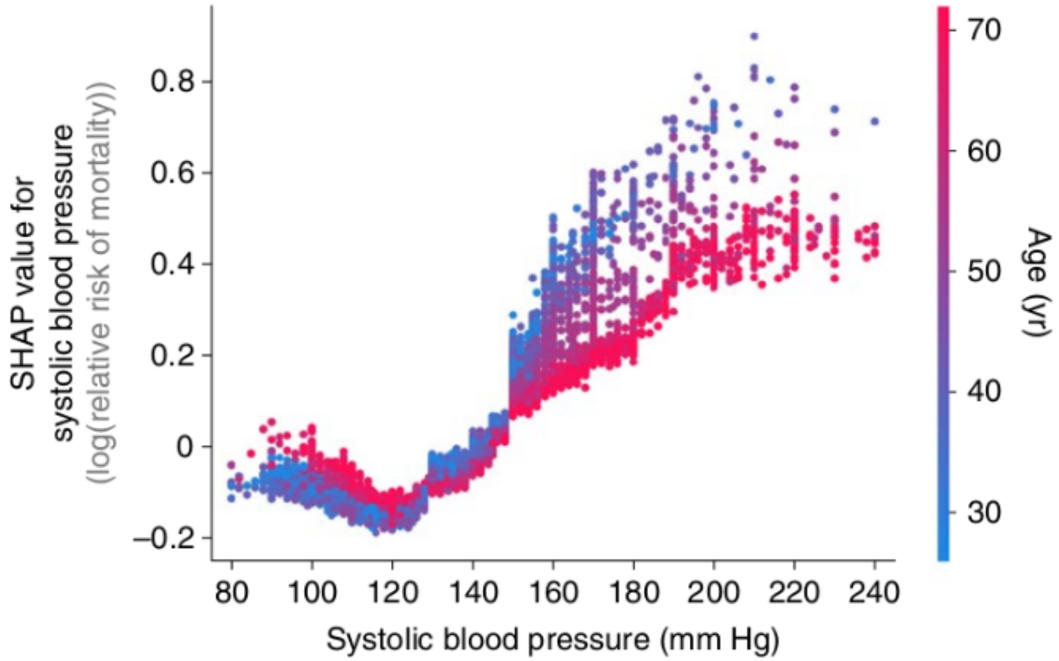


# SHAP dependence plots

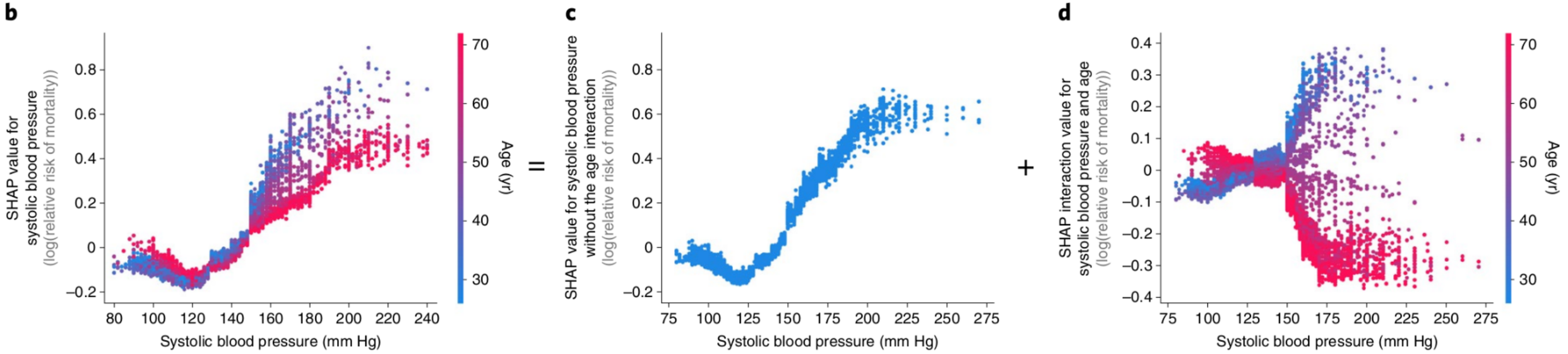
Complements **partial-dependence plots**; shows how feature attribution varies with feature value.

They can be used to see how a feature's attributed importance changes as its value varies.

Vertical dispersion captures the interaction effects in the model.



# Feature interaction (with TreeSHAP)



# How to evaluate the model on a subset of features?

$f_x$(S) should resemble the value of $f(x)$ when we only know the features in S.

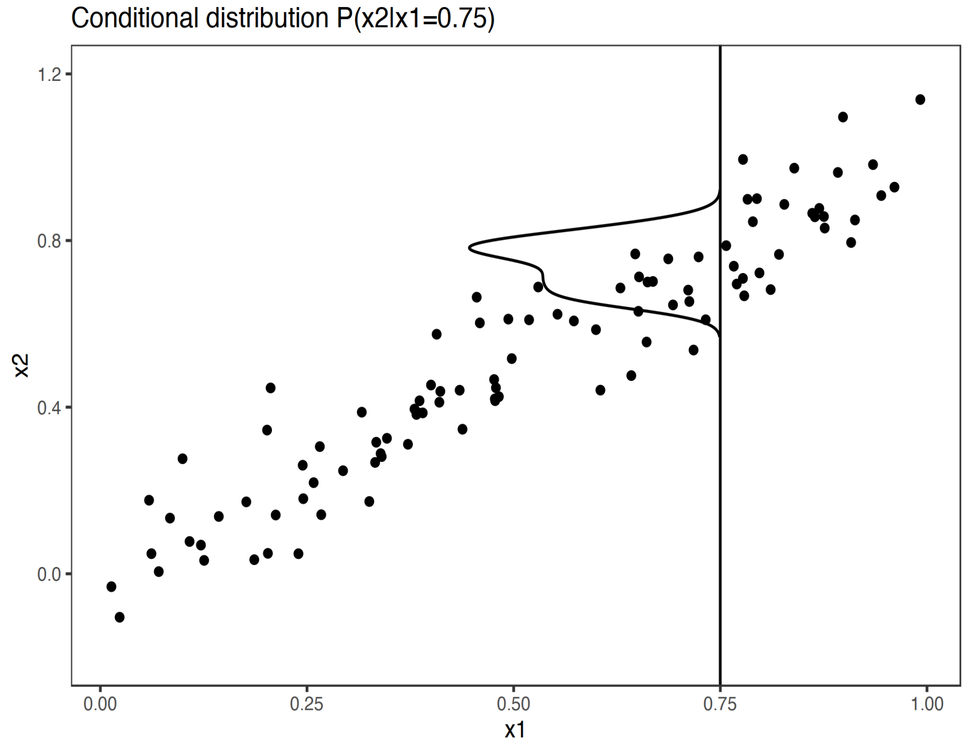
- **Observational/Conditional** approach:

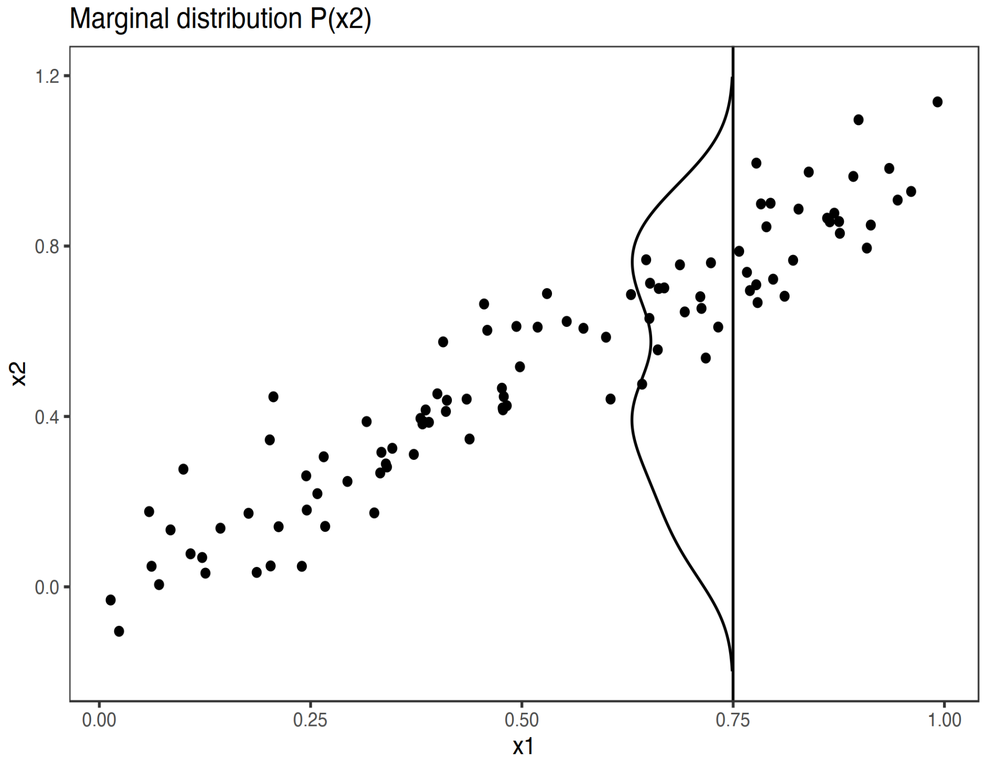
$$E[f(\boldsymbol{x})|\boldsymbol{x}_S = \boldsymbol{x}_S^*] = \int f(\boldsymbol{x}_{\bar{S}}, \boldsymbol{x}_S^*)\, p(\boldsymbol{x}_{\bar{S}}|\boldsymbol{x}_S = \boldsymbol{x}_S^*) d\boldsymbol{x}_{\bar{S}}$$

- **Interventional/Marginal** approach:

$$E[f(\boldsymbol{x}_{\bar{S}}, \boldsymbol{x}_S^*)] = \int f(\boldsymbol{x}_{\bar{S}}, \boldsymbol{x}_S^*)\, p(\boldsymbol{x}_{\bar{S}})\, d\boldsymbol{x}_{\bar{S}}$$

# Interventional vs observational



# Interventional vs observational

Possible issues:

- **Observational** approach: $\mathrm{E}[f(\boldsymbol{x})|\boldsymbol{x}_S = \boldsymbol{x}_S^*]$
  - Features not used by the model can take importance > 0
  - Spread of importance between correlated features
  - Conditional distribution is difficult to estimate

- **Interventional** approach: $E[f(\boldsymbol{x}_{\bar{S}}, \boldsymbol{x}_S^*)]$
  - Feature correlation is ignored → the model is evaluated on unrealistic data points

# Interventional vs observational

- D. Janzing et al, 2019
  "*We conclude that unconditional rather than conditional expectations provide the right notion of dropping features*"

- C. Frye et al, 2021
  "*While off-manifold Shapley values can (i) give rise to incorrect explanations, (ii) hide implicit model dependence on sensitive attributes, and (iii) lead to unintelligible explanations in higher-dimensional data, on-manifold explainability overcomes these problems*"

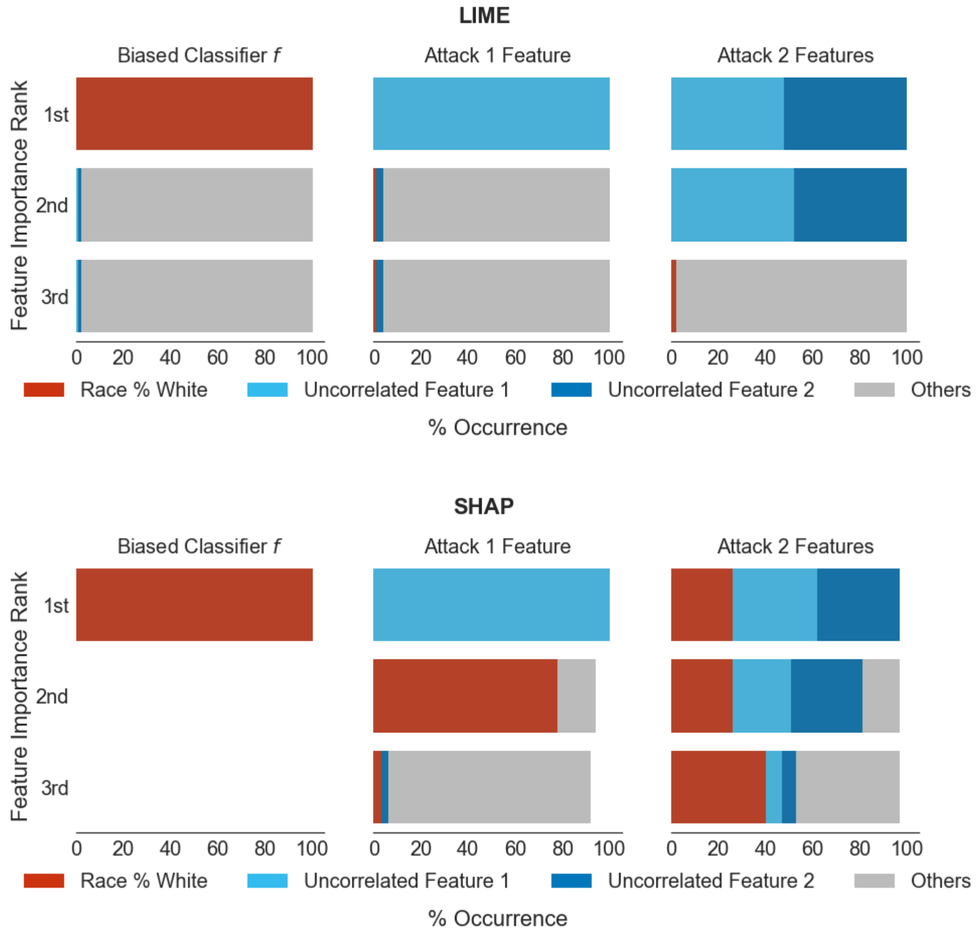
- C. Lundberg et al, 2020
  "*While previous work has argued that one of the two approaches is preferable in general, we argue that the choice is application dependent*"

# Fooling LIME and SHAP with adversarial attacks

It is possible to create **intentionally misleading** interpretations with SHAP, which can hide biases

“Our approach can be used to *scaffold* any biased classifier in such a way that its predictions on the input data distribution still remain biased, but the post hoc explanations of the scaffolded classifier look innocuous.”

Idea: detect if a randomly sampled point is out of distribution or not and use two different classifiers.




Slack et al. Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods, 2020.



# Thanks for the attention!